4D: Solutions to the clustering and PCA exercises

Problem statement and worked resolution

Companion to `04_D_student_exercises.ipynb`

How to use this explanation. The five replacements below are the complete missing code. The student notebook remains unsolved; the answers were executed separately, including all supplied numerical checks. Section numbers follow the notebook's numbered sections where present. Numerical values are rounded and refer to the recorded run, not universal performance claims.

Common interface and data roles

Both exercises use generated data, a fixed 80/20 split, and settings specified before evaluating new rows. There is no supervised response in either fit. A *parameter* such as the requested cluster count is chosen when constructing an estimator; a *fitted attribute* such as learned centers is created by `fit` and often has a trailing underscore. A *method* such as `predict` or `transform` performs an operation using those learned values.

Calling `fit` changes the estimator's learned state. Calling `predict` or `transform` uses that state without relearning it. This distinction matters even for unsupervised models: refitting on the held-out rows would answer a different question from applying a previously learned grouping or representation to them.

1 Exercise 1: assign new rows to learned centers

1.1 Problem and generated data

The supplied code draws 300 two-dimensional rows around $(-3, 0), (3, 0), (0, 4)$, with coordinate standard deviation 0.6. It discards generating labels. The fitting array has shape $(240, 2)$ and the held-out array shape $(60, 2)$. Both features share units, so this exercise deliberately uses them without scaling.

K-means fits centers μ_1, μ_2, μ_3 and minimizes

$$W = \sum_{i=1}^{240} \min_k \|x_i - \mu_k\|^2.$$

K is the number of centers, not the feature dimension, restart count, or number of iterations. Multiple restarts reduce sensitivity to initial centers without proving global optimality.

1.2 TODO 1: create and fit the estimator

Replace the intentional error with:

```
cluster_model = KMeans(n_clusters=3, n_init=20, random_state=23)
cluster_model.fit(X_train)
```

The first line constructs an unfitted estimator. The second learns center coordinates from the 240 fitting rows. `n_init=20` requests 20 initializations; the selected run has lowest inertia among them. `random_state=23` makes this initialization process reproducible in the recorded environment. There is no y argument because the model is unsupervised.

After fitting, `cluster_centers_` has shape $(3, 2)$, `labels_` has length 240, and `inertia_` is a scalar. The recorded inertia is approximately 177.118624. It is a sum on fitting rows, not an accuracy or a probability.

1.3 TODO 2: assign the held-out rows

```
test_groups = cluster_model.predict(X_test)
```

This returns 60 integer IDs. Each is the index of a closest learned center. The supplied check builds a $(60, 3)$ array of squared distances and verifies that the selected IDs minimize each row. This independently connects the method call to its mathematical rule.

The mean held-out squared distance to the assigned center is about 0.625340. For a fair comparison of scale, training inertia per row is $177.118624/240 \approx 0.737994$. The held-out value can be smaller than the fitting average by sampling variation; no principle requires the finite test average to be larger. The raw sum 177.1 and the mean 0.6253 should not be compared directly as if they had the same denominator.

1.4 Answers to the written questions

Exchanging group names 0 and 1 while exchanging the corresponding center names leaves the grouping unchanged. Cluster IDs are identifiers, not ground-truth class labels or a ranking. The partition is about which rows share a group.

Increasing K gives more freedom to reduce squared distances. At K equal to the number of distinct observations, one center per observation can make fitting inertia zero. Therefore inertia alone does not decide an appropriate K. The worked application uses silhouette and stability to add context; those also require interpretation rather than automatic trust.

2 Exercise 2: compress three features to one coordinate

2.1 Problem and generated data

Each row is generated as

$$x = a(1, 2, -1)^T + \epsilon, \qquad a \sim \mathcal{N}(0, 4), \quad \epsilon \sim \mathcal{N}(0, 0.04 I_3),$$

with independent coefficient and noise. The rows lie near one line. There are 240 fitting rows and 60 held-out rows, each with three coordinates. The same original noise scale is used for all features, so the exercise keeps original units.

The population covariance is $4vv^T + 0.04 I_3$ for $v = (1, 2, -1)^T$. Since $\|v\|^2 = 6$, its eigenvalues are $24.04, 0.04, 0.04$. The population first PVE is $24.04/24.12 \approx 0.996683$. This independent calculation predicts a very strong leading direction but does not force the sample result to equal it exactly.

2.2 TODO 3: fit a one-component PCA

```
pca_model = PCA(n_components=1, svd_solver="full")
pca_model.fit(X_pca_train)
```

PCA centers its fitting input automatically, then learns a direction by dense singular value decomposition. It does not standardize feature variances. The learned `mean_` has shape $(3,)$ and `components_` has shape $(1, 3)$. The latter stores one direction as a row. Whitening is off, matching the projection formulas used below.

2.3 TODO 4: compute held-out scores

```
test_scores = pca_model.transform(X_pca_test)
```

The result has shape $(60, 1)$. If v_1 is the learned column direction and $\bar{x}$ the fitting mean, each score is $t_i = (x_i - \bar{x})^T v_1$. The test mean is not subtracted separately: doing so would move the origin and change the representation being evaluated.

2.4 TODO 5: reconstruct the original feature count

```
reconstructed = pca_model.inverse_transform(test_scores)
```

The result has shape $(60, 3)$ and equals $\bar{x} + t_i v_1$ for each row. The supplied identity check uses

```
pca_model.mean_ + test_scores @ pca_model.components_
```

which makes the matrix dimensions explicit: $(60, 1)(1, 3)$ yields $(60, 3)$, then the three-feature mean broadcasts across rows. Inverse transformation does not recover discarded coordinates exactly; it produces the point on the learned affine line with the retained score.

2.5 Results and interpretation

The fitted first variance ratio is approximately 0.996196. Held-out reconstruction MSE per entry is 0.025134, compared with 6.207148 for predicting the fitting mean in all three coordinates. PCA captures the strong line structure, so the improvement over the mean-only baseline is large. These are errors to observed test rows. The exercise does not store clean signal targets and does not claim to measure denoising risk.

If the learned direction changes from v_1 to $-v_1$, each score changes from t_i to $-t_i$; $(-t_i)(-v_1) = t_i v_1$, so the reconstruction is unchanged. Changing only one sign would be wrong. A high PVE establishes retained variance in the fitting geometry, not usefulness for every later prediction task. An unobserved target could depend on a small-variance direction that this one-component representation discards.

Documentation and debugging tips

The official KMeans documentation explains `n_clusters`, `n_init`, `cluster_centers_`, `inertia_`, and new-row assignment. The PCA documentation explains component orientation, transformation, reconstruction, and variance attributes. Read Parameters for constructor choices, Attributes for fitted values, and Methods for callable operations.

1. Replace each `raise NotImplementedError` itself. Code below an unreplaced error is never reached.
2. Run setup and data-generation cells before the TODO cells, then restart and run all after completing the worksheet.
3. Preserve two-dimensional feature arrays: one three-feature observation should have shape $(1, 3)$, not $(3,)$.
4. Fit only on the fitting rows and reuse the same object for new rows; creating a fresh estimator before prediction loses learned state.
5. Interpret `transform` by estimator: K-means returns distances to centers, while PCA returns projected coordinates.
6. Check units and denominators before comparing inertia, per-row squared distance, and per-entry MSE.